

SYSTEM LEVEL POWER OPTIMISATIONS

System Level Design and Modeling

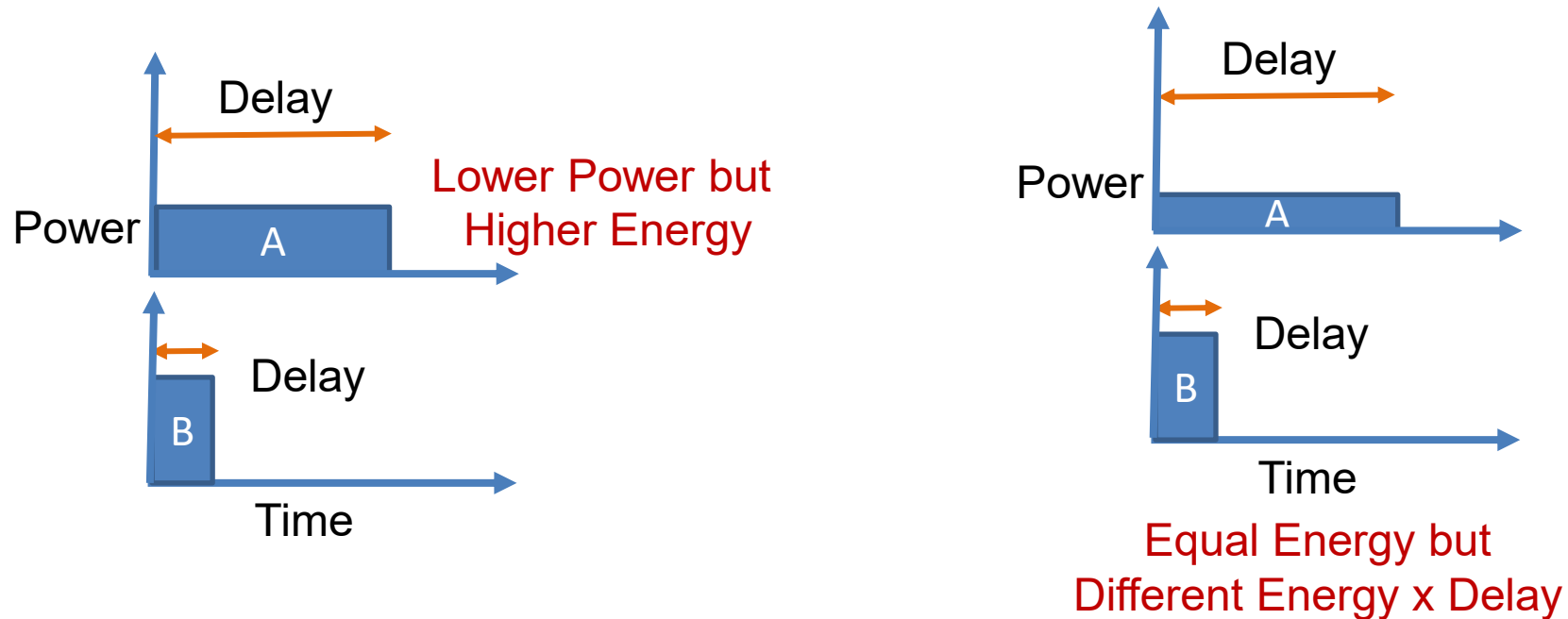
Preeti Ranjan Panda

Department of Computer Science and Engineering

Indian Institute of Technology Delhi

Higher Levels of Abstraction

- **Metrics for Optimization**
 - Average and Peak Power
 - Energy
 - Energy x Delay



Power Optimization Mechanisms

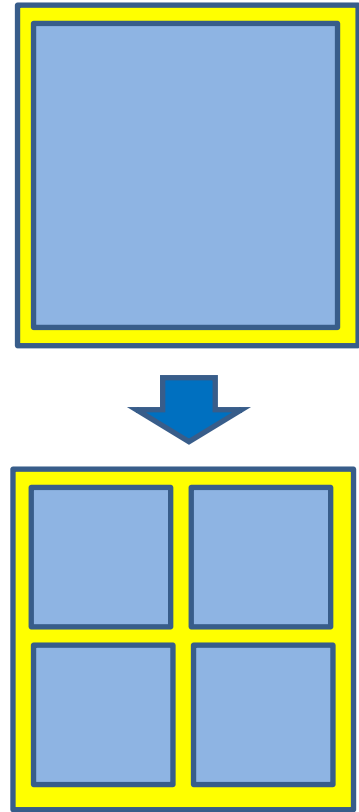
- Basic strategies
 - reduce activity
 - exploit idle times
 - sleep/low power modes
 - voltage/frequency scaling
- Analysis is at higher abstraction levels
 - not enough info. at lower levels

System Level Power Optimizations

- **Processor Design**
- Memory and Cache
- Compiler and Application Software
- Servers and Data Centers
- Example: System Level DVFS

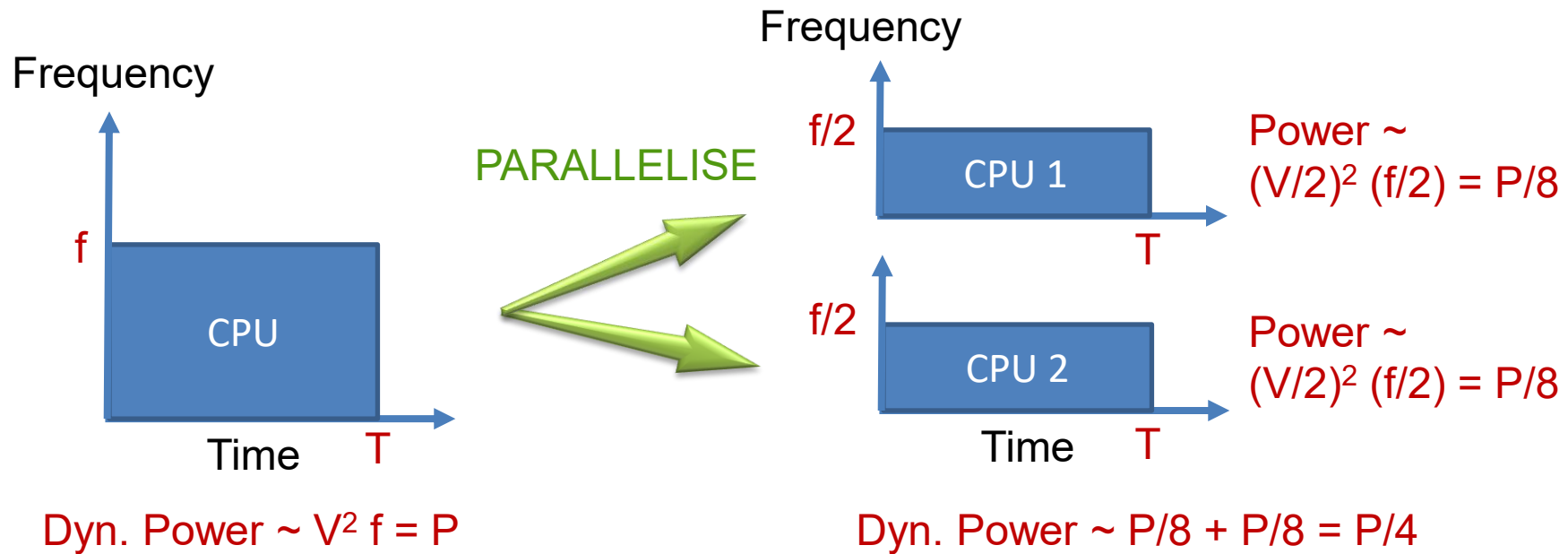
Evolution of Processors

- In the past, towards higher performance
 - ...and higher power
 - higher instruction-level parallelism, but expensive
- Power considerations caused major architecture shift
 - processors no longer getting faster
 - larger number of simpler processors
 - higher throughput

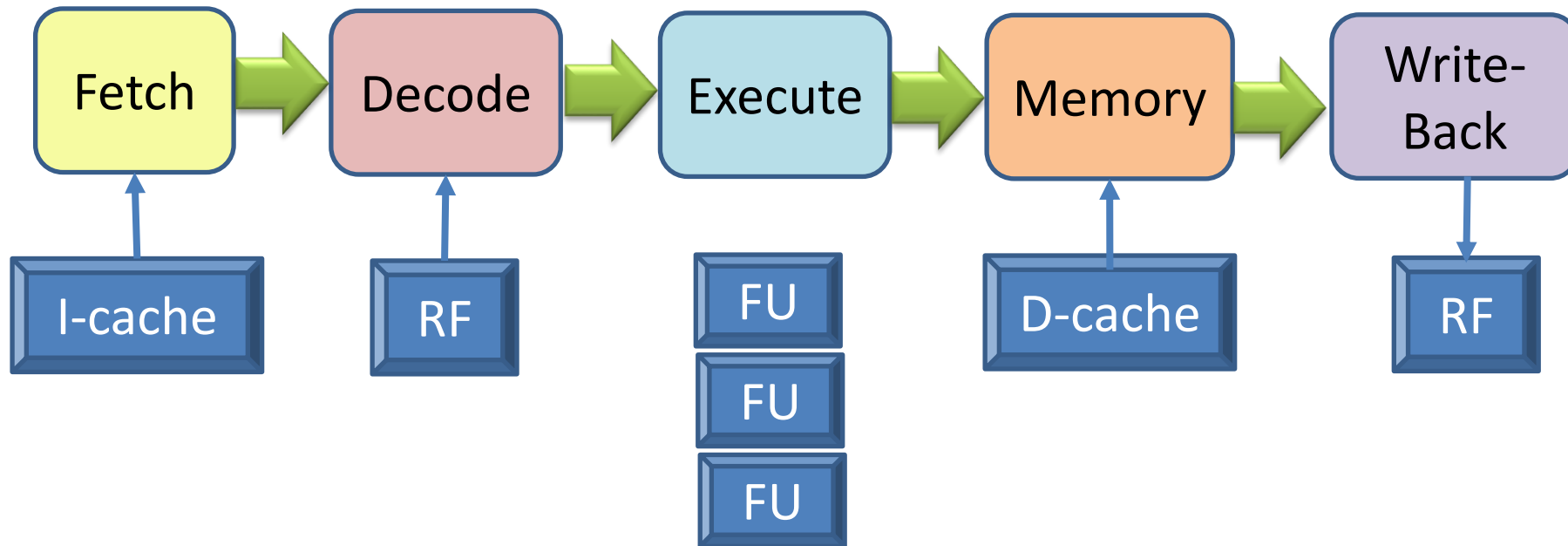


Parallelism and Power: Idealized Argument

- Original Task on single CPU (Voltage V , Frequency f)
- PARALLELISE: 2 CPUs at $f/2$ each (Voltage $V/2$)

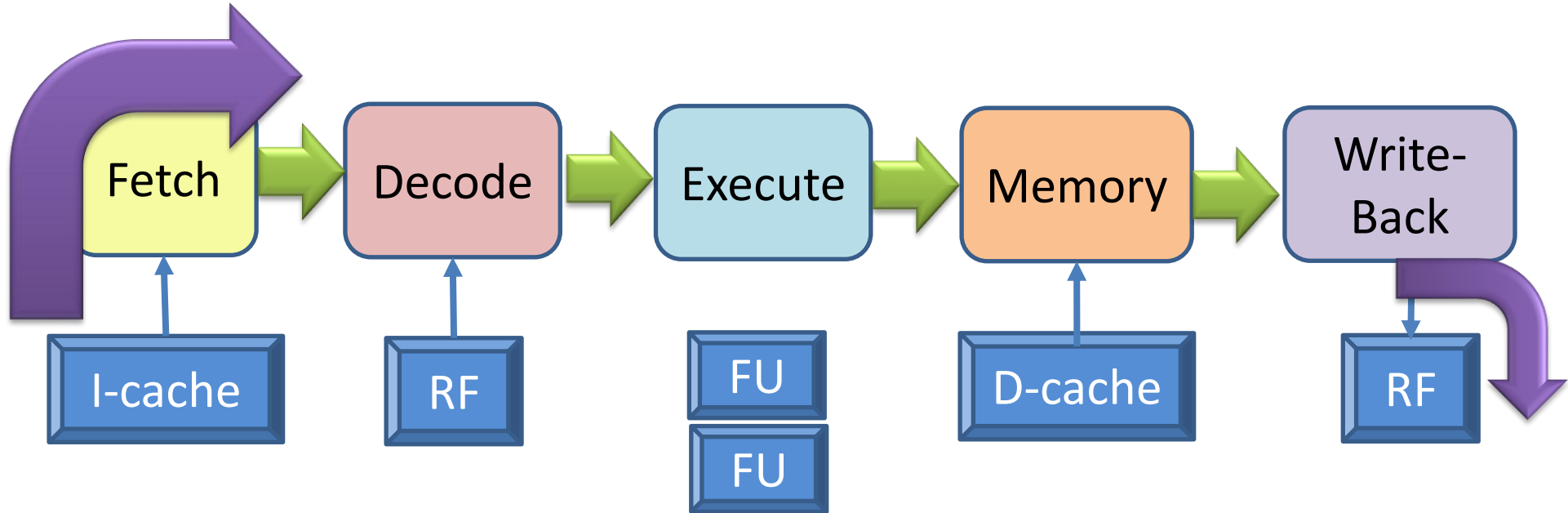


Processor Pipeline



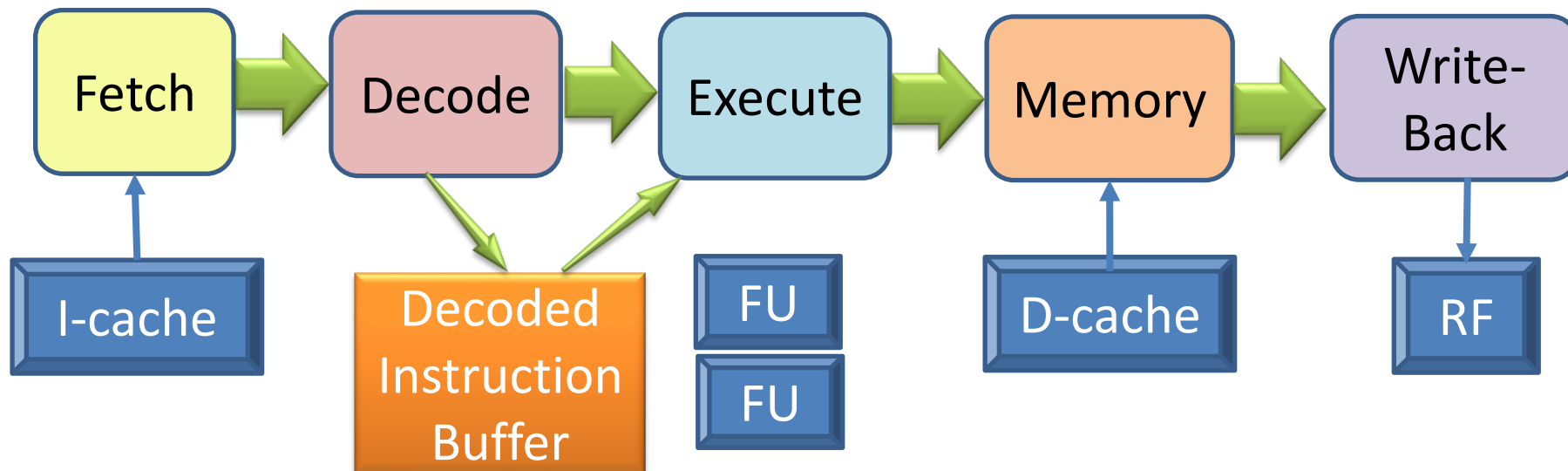
Fetch Gating

- Rate Mismatch: more instructions fetched than committed
 - due to SPECULATION (fetch/execute even before branch cond. known)
- Monitor rates and adjust #inst fetched in front-end



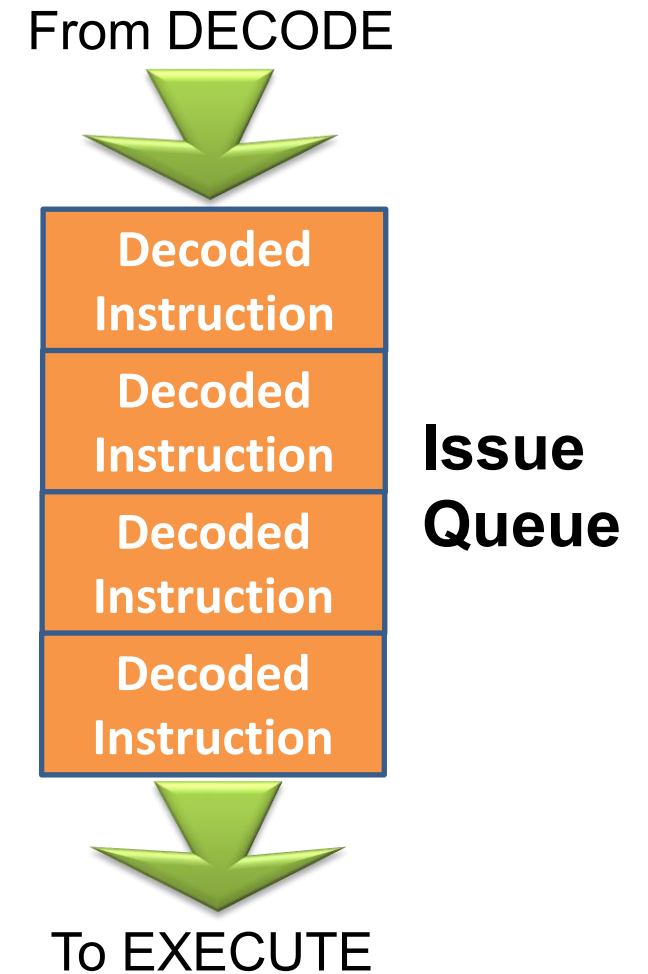
Decode Buffer

- > 90% of program time spent in loops
- Store decoded instructions of loop body in **Buffer**
- Next iteration, read from **Buffer**
 - power saved due to reduced fetches from I-Cache



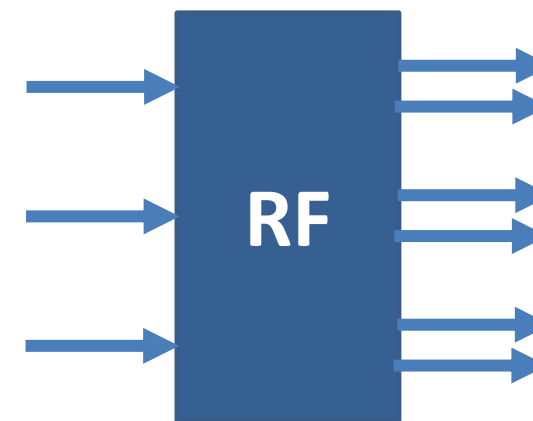
Issue Queue

- Issue Queue holds decoded instructions for execution
 - waiting for operands/FUs to be available
- Associative search in Issue Queue when new operand is available
- Dynamic adaptation of queue size
 - reduces power due to reduced activity
 - performance penalty



Register Files

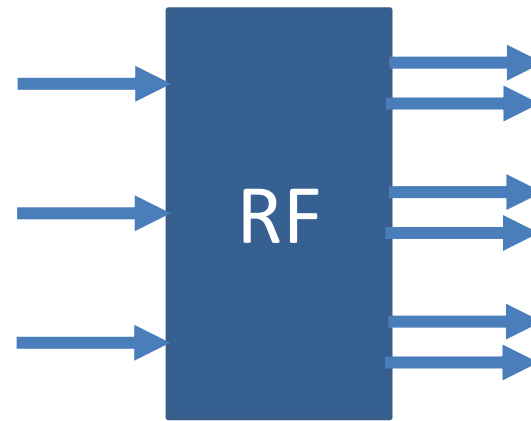
- Important component
 - accessed in almost every instruction
 - 10-15% of processor power
- For executing N independent instructions in a cycle:
 - $2N$ Read Ports, N Write Ports in RF
 - power increases with #ports (RAM cells, bit lines)



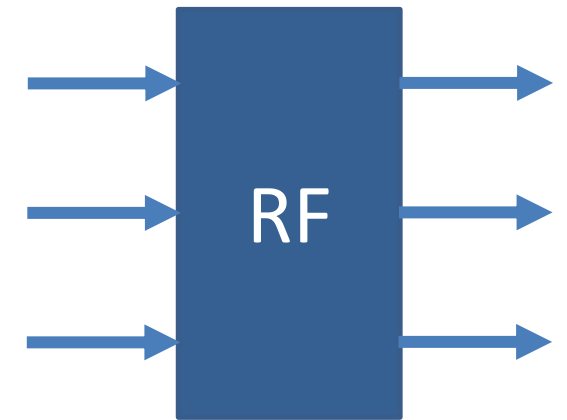
Normal RF
3-way ILP, 9 Ports

Register Files: Reduced Ports

- Base RF: $2N$ Read Ports, N Write Ports in RF
- Reduced #ports
 - lower RF power (but causes conflicts – lower Peak B/W)
 - higher #cycles (increases energy for whole processor)
 - trade-off (extent depends on opt. metric)

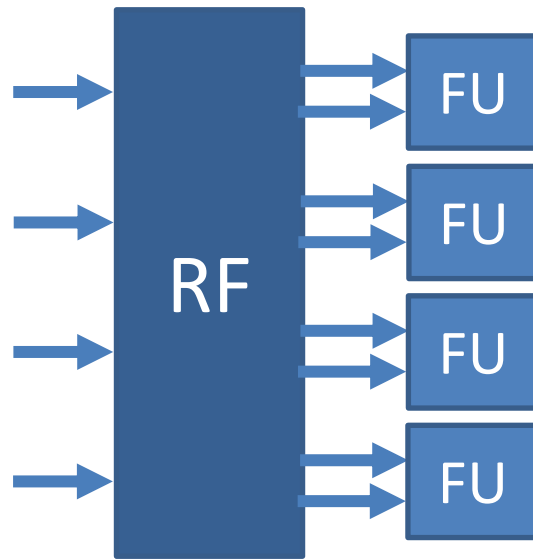


Normal RF
3-way ILP, 9 Ports

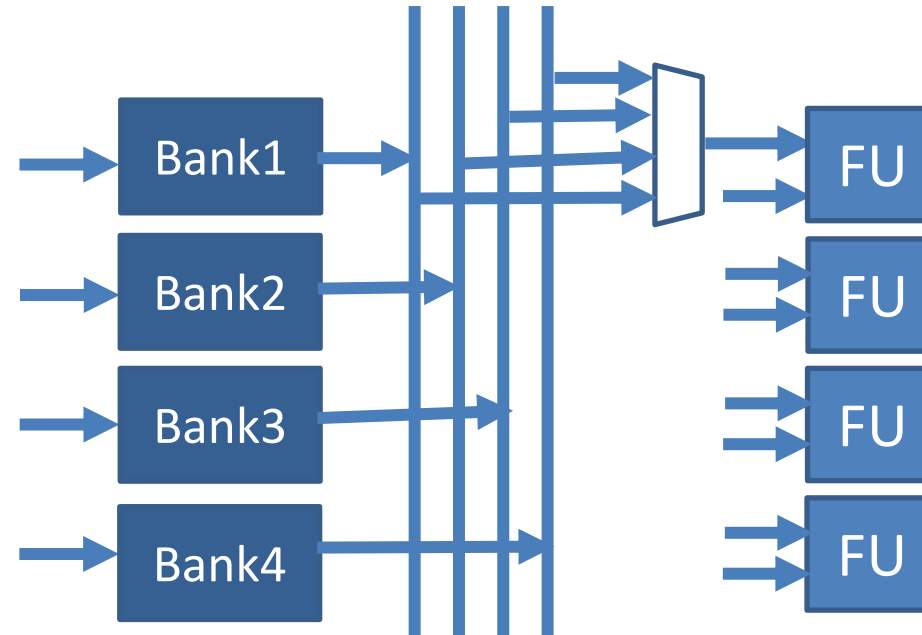


Reduced-port RF
6 Ports. Lower Power,
but Port Conflicts

Restructuring the Register File: Banking

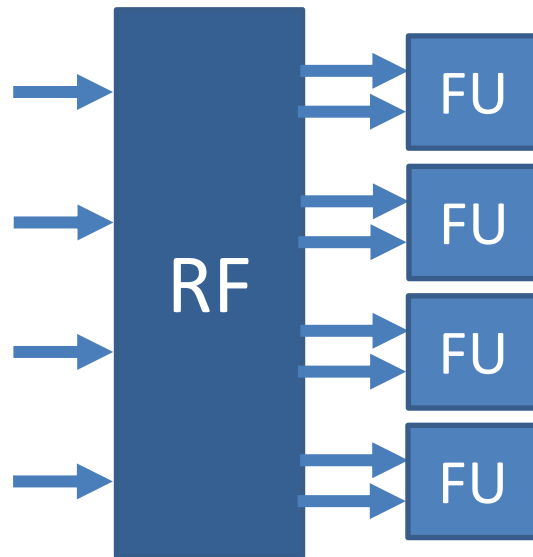


Normal RF
4 FUs, 12 Ports
Full Connectivity

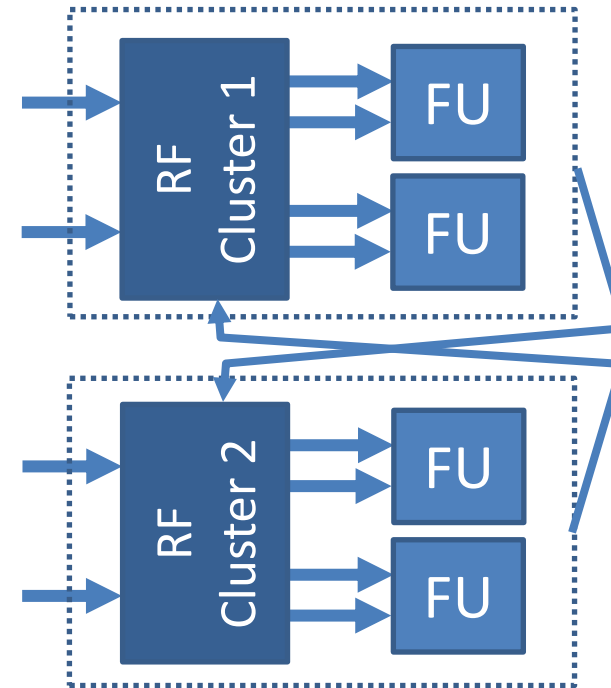


Banked RF
4 FUs, 4 Banks
Max. 1 Read from each Bank

Restructuring the Register File: Clustering

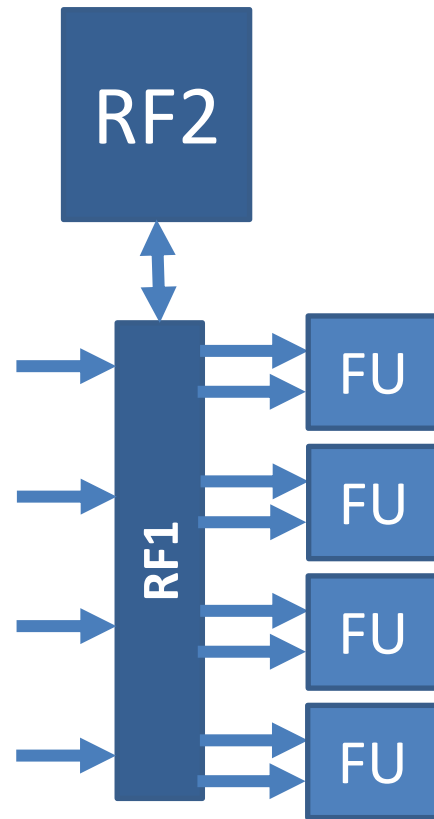


Normal RF
4 FUs, 12 Ports
Full Connectivity



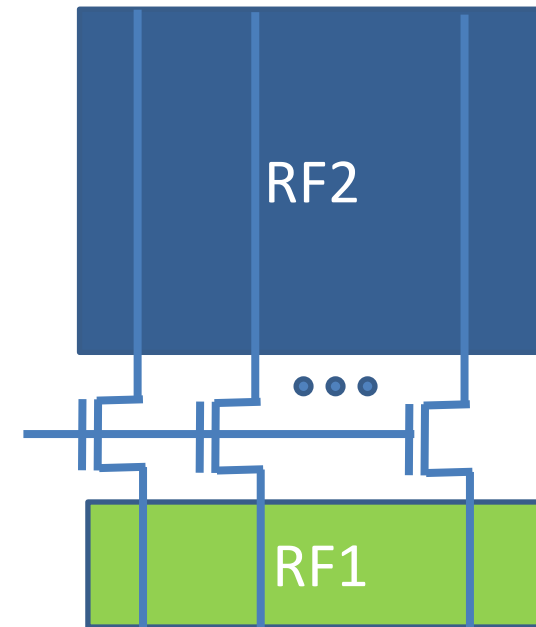
Clustered RF
4 FUs, 2 Clusters
Max. 4 Reads from each cluster
Explicit data transfer across clusters

Restructuring the Register File: Hierarchy



RF Hierarchy. 4 FUs, 2 Levels
First level is small (lower power)

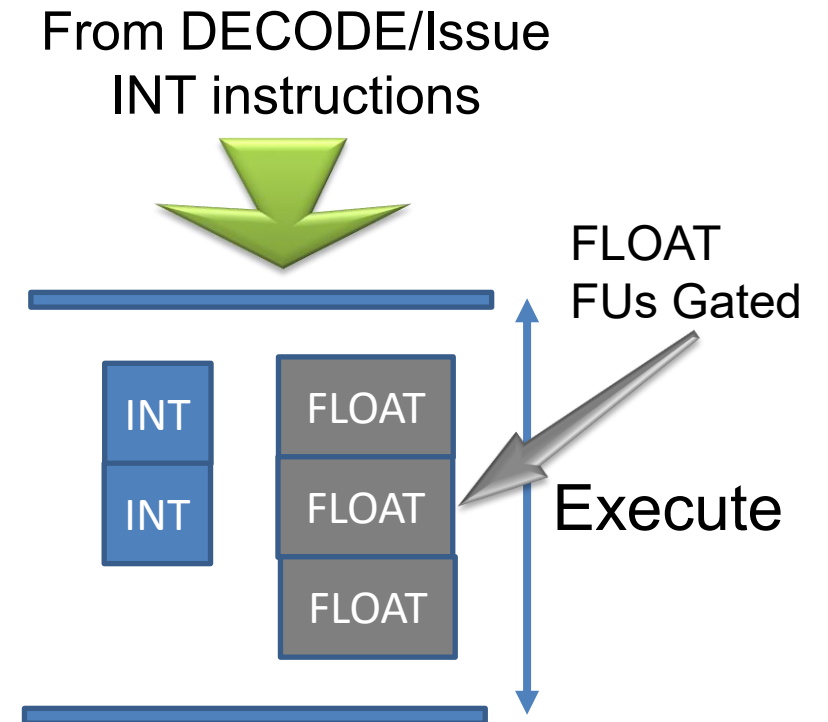
Next level accessed when data not present in first



Implementation

Execution Unit: Clock Gating

- Popular low-level optimization: prevent toggling when FU not in use
- When to gate?
 - Instruction OPCODE known after Decode/Issue
 - Can be used to generate Clock Gate signal for FU
- For what duration to gate?
 - from knowledge of FU



Execution Unit: Datapath Narrowing

- 64-bit or 32-bit datapath often too wide
- Deactivate MSB part of datapath if operands narrow
 - power savings
- Evaluation tricky
 - should be done only if savings are significant compared to overhead of detection

